Backbone de LLM para memória persistente latente

O modelo ideal para este projeto não é o mais inteligente — é o mais dissecável. A escolha do backbone determina a viabilidade de todo o experimento: se a arquitetura esconde os internals atrás de GQA assimétrico, sliding window ou atenção intercalada, isolar o efeito do estado persistente m_t torna-se impossível. Em hardware limitado (Ryzen 5 3600, 16 GB RAM, CPU-only), a viabilidade prática elimina imediatamente qualquer modelo acima de ~1.7B em FP32 no PyTorch. A restrição mais crítica e menos óbvia é que **os três melhores modelos para pesquisa (Pythia, GPT-2, OPT) não estão no Ollama oficial**, enquanto os modelos com cobertura HF+Ollama (SmolLM2, Qwen2.5, LLaMA 3.2) usam arquiteturas modernas com GQA — uma tensão central que define toda a estratégia de escolha.

O que realmente define um bom backbone para este experimento

Este projeto exige um modelo que funcione como **substrato de laboratório**, não como assistente. O estado latente m_t será acoplado ao forward pass em pontos específicos — hidden states entre camadas, KV-cache, ativações intermediárias. Para que o efeito da memória persistente seja isolável, o backbone precisa cumprir condições que nenhum benchmark de chatbot mede.

Multi-Head Attention (MHA) é fortemente preferível a Grouped-Query Attention (GQA). Com MHA, cada head de atenção tem seus próprios pares K,V — injetar no KV-cache é direto. Com GQA, múltiplas query heads compartilham os mesmos K,V heads. Hugging Face APXML Injetar estado em um KV-cache GQA com razão 14:2 (como no Qwen2.5-0.5B) exige decidir como distribuir o sinal injetado entre heads compartilhados, introduzindo um confound metodológico severo.

Atenção sequencial é preferível a atenção paralela. Na maioria dos transformers (GPT-2, Llama, OPT), cada camada processa attention → residual → MLP → residual sequencialmente, permitindo interceptar o sinal entre attention e MLP. Em arquiteturas com atenção paralela (Pythia/GPT-NeoX, Phi-2), attention e MLP recebem a mesma entrada e seus outputs são somados — a injeção precisa acontecer nas fronteiras entre camadas, não dentro delas. Isso não é um bloqueio, mas adiciona complexidade.

Nove critérios formais de avaliação

Cada critério recebe nota de **1 a 5** (5 = melhor). Os pesos refletem a relevância para o projeto:

- **Instrumentabilidade** (peso ×3): acesso a hidden states, attentions, KV-cache, hooks, clareza do código `modeling_*.py`
- **Isolamento da variável memória** (peso ×3): MHA vs GQA, ausência de truques arquiteturais que confundam o efeito, transparência de treinamento

- **Compatibilidade HF ↔ Ollama** (peso ×3): mesmo modelo disponível oficialmente em ambas plataformas, sem conversões manuais
- **Viabilidade local** (peso ×3): cabe em 16 GB RAM com overhead para hidden states em FP32, execução prática em CPU
- **Simplicidade arquitetural** (peso ×2): transformer decoder padrão sem sliding window, soft-capping, atenção intercalada ou truques exóticos
- **Custo operacional** (peso ×2): velocidade de inferência em CPU, viabilidade de iterar rapidamente dezenas de experimentos
- **Reprodutibilidade experimental** (peso ×2): checkpoints intermediários, múltiplas seeds, dados de treino documentados/públicos
- **Adequação para injeção de estado** (peso ×3): formato do KV-cache, pontos de injeção limpos, aceita `inputs_embeds` e `past_key_values`
- **Adequação para comparar full-context vs comprimido** (peso ×2): janela de contexto suficiente para a comparação ser significativa

---

Tabela comparativa dos 10 candidatos

Todos os modelos abaixo funcionam com `AutoModelForCausalLM`, suportam `output_hidden_states=True`, `output_attentions=True`, `use_cache=True` (past_key_values) Hugging Face e permitem `register_forward_hook()` em camadas intermediárias. As diferenças estão na qualidade e simplicidade desse acesso.

Critério (peso)	Pythia-410M	GPT-2 124M	Pythia-1.4B	SmolLM2-1.7B	SmolLM2-360M	Qwen2.5-0.5B	TinyLlama-1.1B	Q
Instrumentabilidade (×3)	5	5	5	4	4	4	4	
Isolamento memória (×3)	5	5	5	4	3	3	3	
Compat. HF↔Ollama (×3)	2	1	2	5	5	5	5	
Viabilidade local (×3)	5	5	4	4	5	5	4	
Simplicidade arq. (×2)	4	5	4	4	3	3	3	
Custo operacional (×2)	5	5	3	3	5	5	4	
Reprodutibilidade (×2)	5	3	5	3	3	2	3	
Injeção de estado (×3)	5	5	5	4	3	2	3	
Full-ctx vs comprimido (×2)	3	2	3	3	3	4	3	
Score ponderado	**100**	**93**	**93**	**89**	**88**	**85**	**83**	

Detalhamento dos candidatos e suas particularidades

Pythia-410M — o substrato de laboratório definitivo

O **EleutherAI/pythia-410m** foi projetado explicitamente para pesquisa de interpretabilidade. GitHub Arquitetura GPT-NeoX com **16 attention heads MHA** (sem GQA), **24 camadas**, hidden_size 1024, DeepWiki deepwiki ativação GELU, RoPE parcial (25% das dimensões). DeepWiki deepwiki Usa atenção paralela (attention + MLP computados em paralelo, estilo GPT-J), deepwiki o que difere do transformer sequencial padrão mas está extensivamente documentado.

O diferencial decisivo: **154 checkpoints intermediários de treinamento** por tamanho, GitHub múltiplas seeds (1-9), GitHub e **8 tamanhos (70M a 12B) treinados nos mesmos dados (The Pile) na mesma ordem.** GitHub +3 Isso permite isolar efeitos de escala e comparar resultados entre tamanhos com rigor impossível em outras famílias. A licença é **Apache 2.0** Hugging Face e os dados de treinamento são públicos.

Em FP32, ocupa ~**2.2 GB** com overhead (1.64 GB só de pesos), atingindo **12-25 tok/s** no PyTorch CPU. Extremamente confortável em 16 GB RAM, com espaço de sobra para hidden states em sequências longas. O acesso é via `model.gpt_neox.layers[i]`.

O ponto fraco: **não está no Ollama oficial**. Existem conversões GGUF comunitárias (TensorBlock para 160M e sizes menores; Hugging Face QuantFactory para 12B), mas pythia-410m e pythia-1.4b **não têm GGUF pré-convertido facilmente acessível**. É possível converter manualmente com `convert-gptneox-hf-to-gguf.py` do llama.cpp (GPT-NeoX é suportado) github GitHub e carregar via Modelfile customizado no Ollama, mas isso adiciona fricção operacional real.

GPT-2 124M — o transformer mais dissecado da história

O **openai-community/gpt2** (124M) é o padrão-ouro de instrumentabilidade. Arquitetura puramente sequencial (attention → residual → MLP → residual), **12 heads MHA**, 12 camadas, hidden_size 768, Amaarora Hugging Face positional embeddings aprendidos (não RoPE), GELU, Hugging Face LayerNorm Hugging Face pré-norma. Nenhum truque moderno — é literalmente o transformer decoder de referência.

Toda ferramenta de interpretabilidade (TransformerLens, baukit, nnsight) suporta GPT-2 primeiro. O `modeling_gpt2.py` do HuggingFace é a implementação de referência usada em centenas de tutoriais. O acesso é via `model.transformer.h[i].attn` e `model.transformer.h[i].mlp`, o mais limpo possível.

Em FP32, ocupa ~**0.7 GB** com overhead. Velocidade de **25-50 tok/s** no PyTorch CPU. Cabe trivialmente na máquina. A licença é MIT modificada.

Pontos fracos: contexto de apenas **1024 tokens** Hugging Face +2 (limitante para comparação full-context vs comprimido), sem checkpoints intermediários, dados de treinamento (WebText) não públicos, e **não está no Ollama oficial** (apenas conversões comunitárias não mantidas).

Pythia-1.4B — Pythia com mais capacidade representacional

Idêntico ao Pythia-410M em arquitetura e design, mas com **24 camadas**, hidden_size 2048, **16 heads MHA**. deepwiki Em FP32, ocupa ~**7.5 GB** com overhead, ainda viável em 16 GB RAM mas com menos margem. Velocidade de **3-6 tok/s** no PyTorch CPU — funcional mas exige paciência para dezenas de experimentos.

Mesmo ponto forte (154 checkpoints, mesmos dados, mesma ordem) GitHub Hugging Face e mesmo ponto fraco (sem Ollama oficial, sem GGUF pré-feito para 1.4B).

SmolLM2-1.7B — o melhor compromisso entre instrumentação e operacionalidade

O **HuggingFaceTB/SmolLM2-1.7B** Hugging Face é o modelo da própria HuggingFace, Ollama

Hugging Face usando arquitetura LlamaForCausalLM com **MHA (32 heads, 32 KV-heads)** huggingface — a exceção na família SmolLM2, já que os menores (135M e 360M) usam GQA. Medium Usa RoPE, RMSNorm, SiLU/SwiGLU, 24 camadas, hidden_size 2048, Hugging Face GitHub contexto de **2048 tokens**.

O diferencial prático: **está oficialmente em ambas plataformas** (`smollm2:1.7b` no Ollama, ollama +2 `HuggingFaceTB/SmolLM2-1.7B` no HuggingFace). Usa `modeling_llama.py`, o arquivo mais bem mantido do Transformers Hugging Face Hugging Face — tudo que funciona para Llama funciona para SmolLM2. Licença **Apache 2.0**. Paper disponível (arXiv:2502.02737) com documentação do mix de dados. Hugging Face +3

Em FP32, ocupa ~**9.2 GB** com overhead. No Ollama (Q4_K_M), apenas ~**1.0 GB**. GGUF Loader GGUF Loader Velocidade de **2-5 tok/s** no PyTorch FP32 e **15-22 tok/s** via Ollama. Usável para iteração, embora mais lento que modelos sub-500M.

Pontos fracos: sem checkpoints intermediários, dados parcialmente documentados (FineWeb-Edu, DCLM, The Stack), Hugging Face +5 apenas 3 tamanhos Hugging Face +2 (e os menores usam GQA, quebrando uniformidade arquitetural).

SmolLM2-360M — iteração rápida com cobertura HF+Ollama

O **HuggingFaceTB/SmolLM2-360M** é como o 1.7B mas com GQA (15 Q-heads, 5 KV-heads), 32 camadas e hidden_size 960. Em FP32, ocupa apenas ~**2 GB** com overhead, rodando a **15-28 tok/s** no PyTorch CPU. Disponível como `smollm2:360m` no Ollama (726 MB). ollama Excelente para prototipar rapidamente.

O problema para este projeto: **GQA com razão 3:1** complica a injeção no KV-cache. Para experimentos focados em hidden states (entre camadas), funciona bem. Para KV-cache injection, o 1.7B (MHA) é muito superior.

Qwen2.5-0.5B — ultra-compacto e moderno, mas GQA extremo

O **Qwen/Qwen2.5-0.5B** tem **494M parâmetros**, 24 camadas, hidden_size 896, Hugging Face contexto nativo de **32K tokens** Hugging Face e está no Ollama como `qwen2.5:0.5b` (398 MB). ollama Usa `modeling_qwen2.py`, implementação limpa com RoPE+YARN, RMSNorm, SwiGLU. Licença **Apache 2.0**.

Mas a razão de GQA é **14:2** — 14 query heads compartilham apenas 2 KV-heads. Hugging Face Isso torna a injeção no KV-cache extraordinariamente complicada. Além disso, usa QKV bias (incomum), arXiv sliding window attention configurável GitHub e dados de treinamento opacos ("até 18T tokens" Ollama sem composição pública). Ollama O contexto longo de 32K é teórico nesta máquina: o KV-cache para 32K tokens consumiria mais RAM do que o disponível.

TinyLlama-1.1B — Llama canônico em escala pequena

O **TinyLlama/TinyLlama-1.1B-Chat-v1.0** replica exatamente a arquitetura LLaMA 2 GitHub ollama com **32 Q-heads, 4 KV-heads (GQA 8:1)**, 22 camadas, hidden_size 2048. Josedavidbaena Está no Ollama como `tinyllama` GGUF Loader (638 MB). ollama Treinado em dados públicos

Medium (StarCoder + SlimPajama, 3T tokens). Medium Medium Checkpoints intermediários disponíveis. GitHub

Ponto forte: se o objetivo é ter um modelo Llama-family no Ollama com equivalente exato no HF, TinyLlama entrega isso. Ponto fraco para este projeto: **GQA 8:1** é bastante agressivo para injeção de KV-cache, e há apenas um tamanho (impossibilita estudos de escala).

Qwen2.5-1.5B — capaz mas com as mesmas limitações de GQA

Similar ao 0.5B em arquitetura (GQA 12:2, 28 camadas, hidden_size 1536), APXML com mais capacidade mas **mesmo problema de injeção de KV-cache**. Em FP32, ocupa ~8.1 GB, ainda viável. No Ollama como `qwen2.5:1.5b` (~1 GB). A janela de 32K é o diferencial teórico. Hugging Face

LLaMA 3.2 1B — moderno mas com origens que complicam

O **meta-llama/Llama-3.2-1B** (1.24B parâmetros) Ollama foi APXML criado por **pruning + knowledge distillation** do LLaMA 3.1 8B. Hugging Face Isso significa que suas representações internas podem não ser "naturais" para um modelo de 1B — foram moldadas pelo modelo maior. Para pesquisa de memória, isso é um confound potencialmente sério: o comportamento dos hidden states pode refletir artefatos da destilação, não propriedades emergentes do treinamento.

Usa GQA APXML (32 Q-heads, 8 KV-heads), 16 camadas, hidden_size 2048, contexto de 128K arXiv (mas impraticável nesta máquina). No Ollama como `llama3.2:1b` (1.3 GB). ollama O acesso no HF é **gated** — requer aceitar a licença Meta, que impõe restrições. Dusty-nv

Phi-2 2.7B — popular mas inviável nesta máquina

O **microsoft/phi-2** tem Hugging Face +2 **MHA real (32/32)**, o que é bom. Mas em FP32, ocupa **~14.6 GB** com overhead, deixando **menos de 2 GB** para OS e hidden states — praticamente inviável para instrumentação com `output_hidden_states=True` em sequências longas. Velocidade estimada de **1-3 tok/s** no PyTorch CPU, tornando iteração experimental dolorosamente lenta.

Além disso: atenção paralela (como Pythia), RoPE parcial (50% das dimensões), Hugging Face biases em todas as projeções, dados de treinamento opacos. Está no Ollama como `phi` (1.6 GB), ollama GGUF Loader onde roda a 9-14 tok/s via Q4 — funcional apenas como baseline, nunca como modelo instrumentado nesta máquina.

Ranking final ponderado

Posição	Modelo	Score	Ponto forte decisivo	Limitação principal
1º	**Pythia-410M**	**100**	Projetado para pesquisa, MHA, 154 checkpoints, 8 tamanhos	Sem Ollama oficial

Posição	Modelo	Score	Ponto forte decisivo	Limitação principal
2º	**GPT-2 124M**	**93**	Arquitetura mais simples e documentada que existe	Contexto 1024, sem Ollama
3º	**Pythia-1.4B**	**93**	Pythia com mais capacidade, mesma qualidade de pesquisa	Sem Ollama, mais lento
4º	**SmolLM2-1.7B**	**89**	MHA + HF + Ollama oficial, modelo da HuggingFace	Mais lento em PyTorch FP32
5º	**SmolLM2-360M**	**88**	Muito rápido, HF+Ollama	GQA 15:5 complica KV-cache
6º	**Qwen2.5-0.5B**	**85**	Ultra-compacto, rápido, Ollama oficial	GQA 14:2 extremo
7º	**TinyLlama-1.1B**	**83**	Llama canônico em HF+Ollama	GQA 8:1
8º	**Qwen2.5-1.5B**	**78**	Capaz, contexto longo teórico	GQA 12:2, dados opacos
9º	**LLaMA 3.2 1B**	**75**	Popular, Ollama oficial	Destilado de 8B, gated
10º	**Phi-2 2.7B**	**71**	MHA real, MIT license	Inviável em FP32 nesta máquina

Recomendação prática final

Se eu fosse começar hoje nesta máquina, começaria com SmolLM2-1.7B

É o único modelo que combina simultaneamente: **MHA real** (32/32, injeção de KV-cache direta), presença oficial no Ollama (`smollm2:1.7b`), implementação nativa via `modeling_llama.py` no HuggingFace, GitHub licença Apache 2.0, e viabilidade em 16 GB RAM (~9.2 GB em FP32 com overhead). A velocidade de 2-5 tok/s no PyTorch FP32 não é rápida, mas permite iteração real. No Ollama com Q4, roda a 15-22 tok/s GGUF Loader GGUF Loader — confortável para baseline. **É o ponto de partida que elimina a maior quantidade de fricção operacional.**

O melhor compromisso entre HF instrumentável e baseline em Ollama é SmolLM2-1.7B

Exatamente o mesmo modelo em ambos os lados. A diferença entre FP32 (HF) e Q4_K_M (Ollama) é um confound conhecido e mensurável — muito menor que trocar de modelo inteiro. Isso dá o **paralelo mais limpo possível** entre o caminho experimental e o baseline.

O candidato mais limpo para experimento controlado é Pythia-410M

Se a prioridade absoluta é rigor experimental sobre conveniência operacional, **Pythia-410M no HuggingFace** é imbatível: 154 checkpoints, múltiplas seeds, Hugging Face 8 tamanhos nos

mesmos dados, GitHub +3 MHA puro, Apache 2.0, documentação acadêmica. Para o baseline no Ollama, seria necessário converter manualmente para GGUF via `convert-gptneox-hf-to-gguf.py` do llama.cpp e criar um Modelfile customizado. Isso é **viável** (GPT-NeoX é suportado no llama.cpp), Ollama GitHub mas adiciona uma etapa técnica que pode introduzir bugs ou diferenças sutis de tokenização.

Estratégia recomendada em fases

Fase 1 — validação do pipeline (semana 1-2): SmolLM2-1.7B em ambos os lados. Monte o pipeline completo: forward instrumentado no HF com injeção de m_t em hidden states e KV-cache, baseline com full-context no Ollama, scripts de comparação. O objetivo é ter o loop experimental funcionando antes de otimizar a escolha do modelo.

Fase 2 — experimento controlado (semana 3+): Pythia-410M no HuggingFace para os experimentos de injeção propriamente ditos. Aqui, o Ollama baseline é opcional — a comparação pode ser feita inteiramente no PyTorch, comparando forward com m_t injetado vs forward com contexto completo reinserido. Pythia-410M roda a 12-25 tok/s em FP32, rápido o suficiente para centenas de rodadas experimentais.

Fase 3 — generalização: Repetir achados-chave em Pythia-160M (menor) e Pythia-1.4B (maior) para verificar se o efeito da memória persistente escala. Depois validar em SmolLM2-1.7B para confirmar que o resultado não é específico da arquitetura Pythia.

O ideal é usar o mesmo backbone em ambos os lados — mas não a qualquer custo

Usar o **mesmo modelo** no HF e no Ollama elimina o confound mais grave: atribuir ao mecanismo de memória uma diferença que na verdade é efeito da troca de arquitetura. O SmolLM2-1.7B permite isso imediatamente. Porém, se a qualidade do experimento exigir um modelo mais instrumentável (Pythia), aceitar modelos diferentes é preferível a comprometer o design experimental. A estratégia de mitigação é simples: validar os achados principais em ambas as famílias de modelos. Se o efeito de m_t aparece tanto em Pythia-410M quanto em SmolLM2-1.7B, ele é provavelmente real e não artefato da arquitetura.

Resumo executivo das escolhas

Papel	Modelo	Plataforma	Justificativa
Backbone instrumentado (HF)	Pythia-410M	HuggingFace PyTorch	MHA, 154 checkpoints, projetado para pesquisa, 12-25 tok/s CPU
Baseline full-context (Ollama)	SmolLM2-1.7B	Ollama `smollm2:1.7b`	Oficial, MHA, Q4 rápido (15-22 tok/s)
Backbone unificado (HF+Ollama)	SmolLM2-1.7B	Ambos	Melhor compromisso quando mesma identidade é crítica
Iteração rápida / protótipos	Pythia-160M ou GPT-2 124M	HuggingFace	Sub-1GB, 25-80 tok/s, feedback instantâneo

Papel	Modelo	Plataforma	Justificativa
Validação de generalização	SmolLM2-1.7B (se partiu de Pythia)	HuggingFace	Confirma que resultado não é específico de GPT-NeoX